

Architecture Design: Serverless Flight Data ETL Pipeline on AWS

Tyler Venner

September 9, 2025

1 System Overview

This pipeline automates daily ingestion, feature transformation, and storage of flight pricing and macroeconomic indicator data on AWS. To minimize infrastructure maintenance and operating costs, the system executes as a containerized serverless job orchestrated by Amazon EventBridge, processing raw API responses directly in memory and persisting partitioned Parquet files to Amazon S3 for ad-hoc SQL querying via Amazon Athena.

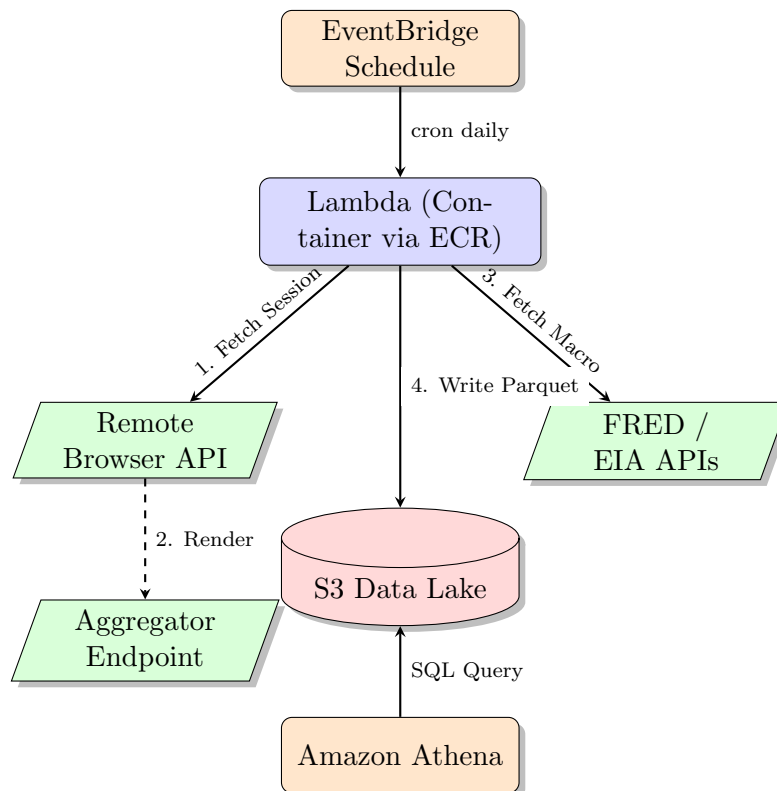


Figure 1: End-to-end data flow and AWS service integration.

Component	Service / Technology	Role in Pipeline
Scheduler	Amazon EventBridge	Daily cron trigger firing invocation payload
Execution Engine	AWS Lambda (Docker, ECR)	Orchestrates scraping, transformations, and writes
Anti-Bot Management	Bright Data Scraping Browser	Remote browser session handling and fingerprint rotation
Storage Layer	Amazon S3	Hive-partitioned data lake storing compressed Parquet
Query Engine	Amazon Athena	Presto/Trino SQL interface over S3 partitions

Table 1: Infrastructure components and service mapping.

2 Hybrid Ingestion Strategy

Scraping dynamic flight aggregators requires executing complex JavaScript and bypassing anti-bot fingerprinting. Rather than running a full browser instance inside Lambda—which introduces heavy memory footprints, binary size constraints, and cold-start latency—the pipeline uses a hybrid two-tier ingestion pattern:

1. **Remote Session Initialization:** Lambda connects to a remote managed Chromium instance via WebSocket using Selenium. The remote service manages residential proxy rotation and passes Cloudflare/Akamai bot challenges to extract transient session tokens and cookies.
2. **Direct Internal API Ingestion:** Once auth tokens are extracted, Lambda terminates the remote browser session and queries the aggregator’s internal REST endpoints directly via HTTP. This fetches raw payload batches in seconds without rendering overhead.
3. **Macroeconomic Ingestion:** In parallel, Lambda fetches jet fuel spot prices (EIA), foreign exchange rates (USD/JPY), and CPI metrics (FRED) to contextualize price trends.

Listing 1: Remote session initialization via Selenium WebDriver.

```

1 import os
2 from selenium import webdriver
3 from selenium.webdriver.chrome.options import Options
4
5 def get_session_cookies(search_url: str) -> dict:
6     remote_url = os.environ["BRIGHTDATA_REMOTE_URL"]
7
8     options = Options()
9     options.set_capability("brd:block_robots", False)
10
11     driver = webdriver.Remote(command_executor=remote_url, options=options)
12     try:
13         driver.get(search_url)
14         # Extract dynamic cookies and authorization headers
15         cookies = {c["name"]: c["value"] for c in driver.get_cookies()}
16         return cookies
17     finally:
18         driver.quit()

```

3 Transformation & Feature Pipeline

The raw JSON payload from the internal API is flattened and transformed in-memory using `pandas`. Processing logic is split into raw parsing (`parser.py`) and feature calculation (`transformer.py`).

Category	Feature	Description / Definition
Market Position	Price Rank	Ordinal index of flight price on route for current scrape date.
	Price Percentile	Normalized rank: $P_p = (\text{rank} - 1) / (N - 1)$ where $N = \text{total route flights}$.
	Delta from Min	Absolute difference from cheapest option: $\Delta_p = p_i - p_{\min}$.
Temporal	Lead Time	Days between scrape timestamp and departure date.
	Departure Window	Period categorization (Early Morning, Morning, Afternoon, Evening).
	Holiday Proximity	Days until the nearest US/Japan national holiday via <code>holidays</code> .
Efficiency	Layover Ratio	$R_{\text{layover}} = T_{\text{total_layover}} / T_{\text{total_flight}}$.
	Cost per Flight Minute	$C_{\text{minute}} = \text{Price} / T_{\text{total_flight}}$.
Macro Factors	Fuel Price	Spot price of US Gulf Coast Kerosene-Type Jet Fuel (EIA).
	FX Rate	Real-time USD/JPY exchange rate.

Table 2: Engineered features for downstream flight price modeling.

Listing 2: Sample schema of final transformed record.

```

1 {
2   "flight_id": "374d7d32da51d15b767df69c64e55aef",
3   "route_name": "TYO_to_USA",
4   "scrape_timestamp": "2025-09-04T20:38:18.438Z",
5   "price_usd": 542.50,
6   "marketing_airline": "ZIPAIR",
7   "stops": 0,
8   "total_flight_minutes": 842,
9   "layover_ratio": 0.0,
10  "cost_per_air_minute": 0.644,
11  "origin": "NRT",
12  "destination": "SFO",
13  "days_to_departure": 5,
14  "price_percentile": 0.072,
15  "price_delta_cheapest": 0.0,
16  "days_to_next_holiday": 6,
17  "usd_jpy_rate": 148.41,
18  "jet_fuel_usd_gal": 2.12
19 }
```

4 Storage Architecture & Query Layer

Data persistence is structured around cost-effective, columnar analytical queries:

- **Serialization:** The processed DataFrame is converted to Apache Parquet format in-memory using `pyarrow` with Snappy compression, reducing storage size and scan I/O.
- **Hive Partitioning:** Files are uploaded to S3 using date-based prefixes (`s3://<bucket>/flights/scrape_date=YYYY-MM-DD/data.parquet`). Athena limits scan ranges to specific partition keys, preventing full bucket scans.

Listing 3: Athena DDL for partitioned external table.

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS flight_lake.processed_flights (  
2     flight_id STRING,  
3     route_name STRING,  
4     scrape_timestamp TIMESTAMP,  
5     price_usd DOUBLE,  
6     marketing_airline STRING,  
7     stops INT,  
8     total_flight_minutes INT,  
9     layover_ratio DOUBLE,  
10    cost_per_air_minute DOUBLE,  
11    origin STRING,  
12    destination STRING,  
13    days_to_departure INT,  
14    price_percentile DOUBLE,  
15    price_delta_cheapest DOUBLE,  
16    days_to_next_holiday INT,  
17    usd_jpy_rate DOUBLE,  
18    jet_fuel_usd_gal DOUBLE  
19 )  
20 PARTITIONED BY (scrape_date STRING)  
21 STORED AS PARQUET  
22 LOCATION 's3://flight-data-lake-prod/flights/';
```

5 Operational Considerations

- **Compute Limits:** The Lambda execution is configured with 1024 MB RAM and a 5-minute timeout. Memory allocation is sized for `pyarrow` and `pandas` DataFrame manipulation, keeping memory usage well below the provisioned ceiling.
- **Cost Optimization:** The hybrid extraction approach restricts remote browser execution to approximately 10–15 seconds per run (solely for token retrieval), minimizing paid proxy browser connection hours.
- **Idempotency:** Ingestion runs generate deterministic partition keys based on UTC execution dates. Re-running the pipeline on a given day overwrites the date partition without introducing duplicate records into downstream Athena datasets.